8.15 Arrays

An **array** is a data structure. It allows you store multiple pieces of data in one structure with one identifier. In an array, each data item must be of the same data type. If it stores integers, then all values must be integers. If it stores strings, then all values must be strings.

KEY WORDS

array: a data structure where you can store multiple data items, of the same data type, under one identifier.

1-dimensional array: an array that has only one index.

index: the number of the space in the array.

1-dimensional arrays

A **1-dimensional array** has just one row of data.

The best way to visualise an array is with a table:

Index	0	1	2	3	4
Data	10	5	90	26	87

This array has 5 spaces. Each space has an **index**. In this array the first data item is in position 0, the data value is 10. In the second array space (index 1), the number 5 is stored.

Arrays can be 0-indexed or 1-indexed. This depends on the programming language that you use. Some arrays start with 0 as the first space. Some arrays start with 1 as the first space.

Arrays use brackets after the identifier to indicate the index you want to access. For example, `Array[0]` is accessing the first element in the array named `Array`. `MyData[3]` is accessing the fourth element in the array named `MyData`.

Putting data into an array

You need to know the array identifier and the position where you want to store the data.

Example 1

Store the colour `"red"` in the first position of the array named `Colour`:

```
Colour[0] ← "red"
```

VB.NET

```
Dim colour(0) As String
colour(0) = "red"
```

Python

```
colour = ["",""]
colour[0] = "red"
```

Java

```
public static void main(String args[]){
    String[] colour = new String[1];
    colour[0] = "red";
}
```

Example 2

Store the colour `"yellow"` in the second position of the array named `Colour`:

```
Colour[1] ← "yellow"
```

VB.NET

```
Dim colour(1) As String
colour(1) = "yellow"
```

Python

```
colour = ["",""]
colour[1] = "yellow"
```

Java

```
public static void main(String args[]){
    String[] colour = new String[2];
    colour[1] = "yellow";
}
```

Example 3

Store the number `20` in the sixth position of the array named `Numbers`:

```
Numbers[5] ← 20
```

VB.NET

```
Dim numbers(9) As String
numbers(5) = 20
```

Python

```
numbers = [0,0,0,0,0,0,0,0,0,0]
numbers[5] = 20
```

Java

```
public static void main(String args[]){
    Integer[] numbers = new Integer[10];
    numbers[5] = 20;
}
```

Getting data out of an array

To access data in an array you need to know the identifier and the position of the data you want. This will be a value, so you need to do something with this value, e.g. store it in a variable.

Example 1

Output the first value in the array `Colours`:

```
OUTPUT(Colours[0])
```

VB.NET

```
Console.WriteLine(Colours(0))
```

Python

```
colours = ['red']
print(colours[0])
```

Java

```
public static void main(String args[]){
    String[] colours = new String[10];
    System.out.println(colours[0]);
}
```

Example 2

Store the second value in the array `Colours` in a variable:

```
TheColour ← Colours[1]
```

VB.NET

```
theColour = colours(1)
```

Python

```
colours = ['red','yellow']
theColour = colours[1]
```

Java

```
public static void main(String args[]){
    String[] colours = new String[10];
    colours[0] = "red";
    colours[1] = "yellow";
    String theColour = colours[1];
}
```

Example 3

Add 10 to the third value in the array Numbers:

```
Value ← 10 + Numbers[2]
```

VB.NET

```
value = 10 + numbers(2)
```

Python

```
numbers = [0,1,2,3,4]
value = 10 + numbers[2]
```

Java

```
public static void main(String args[]){
    Integer[] numbers = new Integer[5];
    numbers[0] = 0;
    numbers[1] = 1;
    numbers[2] = 2;
    numbers[3] = 3;
    numbers[4] = 4;
    Integer value = 10 + numbers[2];
}
```

Using variables as indices

The index in the array might be a variable that stores a number.

Example

Ask the user which array element to output from the array colours:

```
OUTPUT("Enter the array element you want to output")
INPUT ToOutput
OUTPUT(Colours[ToOutput])
```

VB.NET

```
Dim colours(9) As String
colours(0) = "red"
colours(1) = "yellow"
colours(2) = "black"
colours(3) = "green"
Console.WriteLine("Enter the array element you want to output")
Console.WriteLine(colours(Console.ReadLine))
```

Python

```
colours = ['red','yellow','black','green']
print(colours[int(input("Enter the array element you want to output"))])
```

Java

```
public static void main(String args[]){
    String[] colours = new String[10];
    colours[0] = "red";
    colours[1] = "yellow";
    colours[2] = "black";
    colours[3] = "green";
    System.out.println("Enter the array element you want to output");
    Scanner scanner = new Scanner(System.in);
    Integer choice = Integer.parseInt(scanner.nextLine());
    System.out.println(colours[choice]);
}
```

Using iteration to read and write

If you have a set of values in an array you can use iteration to loop through each of the elements in turn. For example, you might want to output all of the values one at a time. You could add together all of the values in an array and output the total. You might want to take 10 values in from the user and store each one in the array.

These are all best done using a count-controlled loop. This is because you usually know how many values you want to enter, or how many values there are in the array that you want to work through.

Example 1

Output all 10 elements in the array `Colours`:

```
FOR Count ← 0 TO 9
    OUTPUT(Colours[Count])
NEXT Count
```

VB.NET

```
Dim colours(9) As String
colours(0) = "red"
colours(1) = "yellow"
colours(2) = "black"
colours(3) = "green"
colours(4) = "blue"
colours(5) = "white"
colours(6) = "orange"
colours(7) = "purple"
colours(8) = "grey"
colours(9) = "maroon"
For count = 0 To 9
    Console.WriteLine(colours(count))
Next
```

Python

```
colours = ['red','yellow','black','green','blue','white','orange','purple',
    'grey','maroon']
    for count in range(0, 10):
        print(colours[count])
```

Java

```
public static void main(String args[]){
    String[] colours = new String[10];
    colours[0] = "red";
    colours[1] = "yellow";
    colours[2] = "black";
    colours[3] = "green";
    colours[4] = "blue";
    colours[5] = "white";
    colours[6] = "orange";
    colours[7] = "purple";
    colours[8] = "grey";
    colours[9] = "maroon";
    for(Integer count = 0; count < 10; count++){
        System.out.println(colours[count]);
    }
}
```

Example 2

Ask the user to input 20 numbers and store each in the array `Numbers`:

```
FOR counter ← 0 TO 19
    OUTPUT("Enter a number")
    INPUT Numbers[Counter]
NEXT counter
```

VB.NET

```
Dim numbers(20) As Integer
For count = 0 To 19
    Console.WriteLine("Enter a number")
    numbers(count) = Console.ReadLine()
Next
```

Python

```
numbers = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
for count in range(0, 20):
    numbers[count] = int(input("Enter a number"))
```

Java

```
public static void main(String args[]){
    Integer[] numbers = new Integer[20];
    Scanner scanner = new Scanner(System.in);
    for(Integer count = 0; count < 20; count++){
        System.out.println("Enter a number");
        numbers[count] = Integer.parseInt(scanner.nextLine());
    }
}
```

Example 3

Searching the values in the array `values` that has 50 values, for the data input by the user:

```
ValueToFind ← INPUT("Enter the value to find")
FOR Count ← 0 TO 49
    IF Values[Counter] = ValueToFind THEN
        OUTPUT("Found it")
    ENDIF
NEXT Count
```

VB.NET

```
Dim values(49) As Integer
'insert code to populate array
Console.WriteLine("Enter the value to find")
Dim valueToFind As Integer = Console.ReadLine
For count = 0 To 49
    Console.WriteLine("Enter a number")
    If values(count) = valueToFind Then
        Console.WriteLine("Found it")
    End If
Next
```

Python

```
values=[]
# insert code to populate array
valueToFind = int(input("Enter the value to find"))
for count in range(0, 50):
    if values[counter] = valueToFind:
        print("Found it")
```

Java

```
public static void main(String args[]){
    Integer[] values= new Integer[50];
    //insert code to populate array

    Scanner scanner = new Scanner(System.in);
    Integer valueToFind = Integer.parseInt(scanner.nextLine());
```

```
        for(Integer count = 0; count < 50; count++){
            System.out.println("Enter a number");
            if(values[count] == valueToFind){
                System.out.println("Found it");
            }
        }
    }
```

2-dimensional arrays

A **2-dimensional array** is best viewed as a table with rows and columns.

KEY WORD

2-dimensional array: an array that has two indices.

Index	0	1	2	3	4
0	10	5	90	26	87
1	3	15	74	62	5
2	7	10	85	4	24

In a 2-dimensional array there are two indices. For example, from the table:

`Position[0, 0]` is `10`.

`Position[0, 2]` is `7`.

`Position[4, 2]` is `24`.

Putting data into an array

You need to know which position, i.e. both indices, the across and the down.

Example 1

Store `"red"` in the first position in the array `Colours`:

```
Colour[0, 0] ← "red"
```

VB.NET

```
colours(0,0) = "red"
```

Python

```
numbers = [[''] * 5 for i in range(10)]
numbers[0][0] = "red"
```

Java

```
public static void main(String args[]){
    String[][] colours = new String[10][10];
    colours[0][0] = "red";
}
```

Example 2

Store `10` in the array `Data`, in element `4` across and `3` down:

```
Data[4, 3] ← 10
```

VB.NET

```
data(4,3) = 10
```

Python

```
numbers = [[0] * 5 for i in range(5)]
numbers[4][3] = 10
```

Java

```
public static void main(String args[]){
    Integer[][] data = new Integer[10][10];
    data[4][3] = 10;
}
```

Getting data out of an array

You need to know both indices to access the data.

Example 1

Output the data in the array `ArrayData`, element `5` across and `1` down:

```
OUTPUT(ArrayData[4, 1])
```

VB.NET

```
Console.WriteLine(arrayData(4,1))
```

Python

```
arrayData = [[0] * 5 for i in range(5)]
print(str(arrayData[4][1]))
```

Java

```
public static void main(String args[]){
    Integer[][] arrayData = new Integer[5][5];
    //insert data to populate array
    System.out.println(arrayData[4][1]);
}
```

Example 2

Access the data in the array colours, in the first element across and the third down:

```
ColourAccessed ← Colours[0, 2]
```

VB.NET

```
colourAccessed = colours(0,2)
```

Python

```
colours = [[''] * 5 for i in range(5)]
colourAccessed = colours[0][2]
```

Java

```
public static void main(String args[]){
    String[][] colours = new String[10][5];
    String colourAccessed = colours[0][2];
}
```

Using variables as indices

Each index can be stored in a variable in the same way as they can be in a 1D array.

Example 1

Output the data in element 4,3:

```
First ← 4
Second ← 3
OUTPUT(ArrayData[First, Second])
```

VB.NET

```
Dim arrayData(9, 9) As String
'insert code to populate array
Dim first As Integer = 4
Dim second As Integer = 3
Console.WriteLine(arrayData(first, second))
```

Python

```
arrayData = [[''] * 5 for i in range(5)]
first = 4
second = 3
print(arrayData[first][second])
```

Java

```
public static void main(String args[]){
    String[][] arrayData = new String[10][10];
    //insert code to populate array
    Integer first = 4;
    Integer second = 3;
    System.out.println(arrayData[first][second]);
}
```

Example 2

Ask the user which element to store in `data`:

```
OUTPUT("Enter dimension 1")
INPUT Index1
OUTPUT("Enter dimension 2")
INPUT Index2
Data ← Array[Index1, Index2]
```

VB.NET

```
Dim arrayData(9, 9) As String
'insert code to populate array
Dim first As Integer
Console.WriteLine("Enter dimension 1")
first = Console.ReadLine
Dim second As Integer
Console.WriteLine("Enter dimension 2")
second = Console.ReadLine
Dim data As Integer
data = arrayData(index1, index2)
```

Python

```
arrayData = [[''] * 10 for i in range(10)]
#insert code to populate array

index1 = int(input("Enter dimension 1"))
index2 = int(input("Enter dimension 2"))
data = arrayData[index1][index2]
```

Java

```
public static void main(String args[]){
    Scanner scanner = new Scanner(System.in);
    String[][] arrayData = new String[10][10];
    //insert code to populate array
    Integer index1 = Integer.parseInt(scanner.nextLine());
    Integer index2 = Integer.parseInt(scanner.nextLine());
    String data = arrayData[index1][index2];
}
```

Using iteration to read and write

Due to the two dimensions, you need two nested loops to read through all the data elements. If you think about the table again, one loop goes through the columns and one loop goes through the rows.

The first loop will check row 1. The column will change from 0, 1, 2, 3, 4. The row will stay the same at 0.

Index	0	1	2	3	4
0	10	5	90	26	87
1	3	15	74	62	5
2	7	10	85	4	24

The first loop will check row 2. The column will change from 0, 1, 2, 3, 4. The row will stay the same at 1.

Index	0	1	2	3	4
0	10	5	90	26	87
1	3	15	74	62	5
2	7	10	85	4	24

It is best to use count controlled loops to go through the array.

It has the structure:

```
FOR row ← first index to last index
    FOR column ← first index to last index
        Code to run
    NEXT row
NEXT count
```

Example 1

`DataArray` has 10 elements by 3 elements. Output all of the elements in the array:

```
FOR Row ← 0 TO 2
    FOR Column ← 0 TO 9
        OUTPUT(DataArray[Column, Row])
    NEXT Row
NEXT Count
```

VB.NET

```
Dim dataArray(2, 9) As Integer
'insert code to populate array

For row = 0 To 2
    For column = 0 To 9
        Console.WriteLine(dataArray(row, column))
    Next
Next
```

Python

```
arrayData = [[''] * 2 for i in range(10)]
#insert code to populate array

for row in range(0, 3):
    for column in range(0, 10):
        print(arrayData[row][column]
```

Java

```
public static void main(String args[]){
    Integer[][] dataArray = new Integer[3][10];
    //insert code to populate array
    for(Integer row = 0; row < 3; row++){
        for(Integer column = 0; column < 10; column++){
            System.out.println(dataArray[row][column]);
        }
    }
}
```

Example 2

Search a 2-dimensional array, with 50 elements by 100 elements, for the value input by the user:

```
OUTPUT("Enter a number to search for")
INPUT SearchValue
FOR Row ← 0 TO 49
    FOR Column ← 0 TO 99
        IF DataArray[Row, Column] = SearchValue THEN
            OUTPUT("Found it at " & Column & " " & Row)
        ENDIF
    NEXT Column
NEXT Row
```

VB.NET

```
Dim dataArray(50, 100) As Integer
'insert code to populate array
Dim searchValue as Integer
Console.WriteLine("Enter a number to search for")
searchValue = Console.ReadLine()
for (row = 0 to 50)
    for(column = 0 to 100)
        if(dataArray(row, column) =searchValue) then
            Console.WriteLine("Found it at " & column & " " & row)
        endif
    next
next
```

Python

```
arrayData = [[0] * 50 for i in range(100)]
#insert code to populate array
searchValue = int(input("Enter a number to search for"))
for row in range(0, 50):
    for column in range(0, 100):
        if arrayData[column][row] = searchValue:
            print("Found it at", str(column), " ", str(row))
```

Java

```
public static void main(String args[]){
    Integer[][] dataArray = new Integer[50][100];
    Scanner scanner = new Scanner(System.in);
    Integer searchValue = Integer.parseInt(scanner.nextLine());
    for(Integer row = 0; row < 50; row++){
        for(Integer column = 0; column < 100; column++){
            if(dataArray[row][column] == searchValue){
                System.out.println("Found it at " + column + " " + row);
            }
        }
    }
}
```

Example 3

Find and output the total of all elements in the each of the first dimensions, in an array of 10 elements by 15 elements:

```
FOR Row ← 0 TO 9
    Total ← 0
    FOR Column ← 0 TO 14
        Total ← Total + TheArray[Row, Column]
    NEXT Column
    OUTPUT("Index " & Row & " has the total " & Total)
NEXT Row
```

VB.NET

```
Dim theArray(9, 14) As Integer
'insert code to populate array
Dim total As Integer = 0
For row = 0 To 9
    total = 0
    For column = 0 To 14
        total = total + theArray(row, column)
    Next
    Console.WriteLine("Index " & row & " has the total " & total)
Next
```

Python

```
theArray = [[0] * 10 for i in range(15)]
#insert code to populate array
total = 0
for row in range(0, 10):
    for column in range(0, 15):
        total = total + theArray[row][column]
    print("Index",row,"has the total",total)
```

Java

```
public static void main(String args[]){
    Integer[][] theArray = new Integer[10][15];
    //insert code to populate array

    Integer total = 0;
    for(Integer row = 0; row < 10; row++){
        for(Integer column = 0; column < 15; column++){
            total = total + theArray[row][column];
        }
        System.out.println("Index " + row + " has the total " + total);
    }
}
```

PROGRAMMING TASK 8.4

The 2-player game of noughts and crosses has a grid of 3 squares by 3 squares (Figure 8.4).

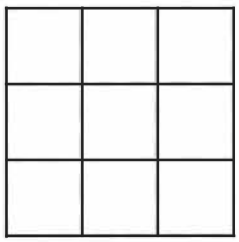


Figure 8.4: A noughts and crosses grid

One player is noughts, the other is crosses.

Each player takes it in turn to select a box to place their nought or cross. They cannot select a box that has already been chosen.

The first player to get three of their symbols in a row (horizontally, vertically or diagonally) wins. If the board is full and no-one has won then it is a draw.

CONTINUED

Getting started

1 Decompose the problem into its inputs, processes and outputs.

2 Work in pairs to discuss how you will alternate between the players.

3 Work in pairs to discuss how you will check if a player has won.

Practice

1 Write the program to ask each player to make one move. Check that they have not selected the same box, if they have, ask them to select another box.

2 Write an algorithm to check if a player has won and either output who has won, or continue playing.

Challenge

1 Write a function for your algorithm to check if a player has won or not. This should check all possible ways of winning and return either: X (crosses has won), O (noughts has won) or C (continue play as no-one has won). Your main program will need to decide whether to end, or continue based on the value returned.

2 Edit your program to allow the user to play multiple games. The player should alternate allowing noughts to go first, and then crosses to go first.

3 Edit your program to allow the user to select how many games they should play. Keep track of how many games each player has won and output who won overall.

Questions

38 Explain the difference between a variable and an array.

39 Explain why the following code will result in an error.

```
MyData[0] ← 1
MyData[1] ← 4
MyData[2] ← "7"
MyData[3] ← "9"
```

40 Write a program to read 10 numbers from the user into a 1-dimensional array named `MyNumbers`.

41 Write a program to add together all 100 elements in a 1-dimensional array named `MyNumbers`.

42 A 2-dimensional array, `MyData`, has 20 elements by 5 elements. Write a function that takes a parameter search value. The function should search `MyData` and return either `TRUE` if the parameters is in `MyData`, or `FALSE` if the parameters is not in `MyData`.